ΑΝΑΦΟΡΑ

Εργασία 2η για το μάθημα των Παράλληλων Συστημάτων

Τζαφέρης Στέφανος 1115202200183

Γαλιατσάτου Ελευθερία 1115202200025

Όλα τα πειράματα πραγματοποιήθηκαν στο linux20 και Linux15 μηχάνηματα της σχολής, όταν δεν υπήρχε κάποιος άλλος χρήστης συνδεδεμένος

Αριθμός πυρήνων : 4

Επεξεργαστής : Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

OS : NAME="Ubuntu" VERSION="20.04.6 LTS (Focal Fossa)"

Έκδοση μεταγλωττιστή : 9.4.0

<u>Άσκηση 2.1</u>

Threads	$n = 10^4$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0005	0.3455	0.3873	0.3492	0.9900
2	0.0007	0.3537	0.1804	1.9600	98.010
4	0.0006	0.3454	0.0983	3.5160	87.844
8	0.0008	0.3493	0.1000	3.4940	43.684

Threads	$n = 10^5$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0041	34.032	34.679	0.9800	98.134
2	0.0071	34.013	17.876	1.9000	95.132
4	0.0042	34.117	9.7804	3.4880	87.208
8	0.0072	34.076	9.6007	3.5500	44.368

Για το πρόβλημα με τον πολλαπλασιασμό πολυωνύμων ακολουθήθηκε ακριβώς η ίδια λογική όπως και με τα pthreads. Μία ενδιαφέρουσα παρατήρηση που δεν την βάλαμε στους πίνακες ,διότι οι διαφορές χρονικά ήταν μικρές, είναι ότι οι χρόνοι ήταν καλύτεροι με την χρήση guided και dynamic scheduling όπου για 4 και 8 threads (n = =10^5) είχαν καλύτερο χρόνο κατά 2-3 δευτερόλεπτα από ότι η static scheduling. Αυτό είναι λογικό αφού για μεγαλύτερο βαθμό πολυωνύμων το workload είναι μεγαλύτερο άρα και η κατανομή όχι ομοιόμορφη, επομένως θέλουμε να είναι δυνατόν όταν κάποιο thread τελειώνει να μπορεί να πάρει άμεσα δουλειά. Να σημειωθεί ότι για static,1 ή για static,10 δηλαδή να παίρνει κυκλικά κάθε thread κομμάτια η απόδοση είναι παρόμοια με του dynamic-guided. Το πρόβλημα υπήρξε μόνο στο σκέτο static. Τελικά κρατήθηκε το schedule guided. Σχετικά με τους χρόνους του πίνακα βλέπουμε λογικές βελτιώσεις και speedup και στις 2 περιπτώσεις. Οι χρόνοι για 1,2,4 threads είναι λίγο πιο γρήγοροι από ότι με Pthreads αλλά για 8 threads βλέπουμε ότι ο χρόνος στην μία περίπτωση αυξάνεται ελάχιστα και στην άλλη μειώνεται ελάχιστα, δεν υπάρχει δηλαδή αισθητή μείωση από 4->8 threads που από την μία είναι λογικό αφού το μηχάνημά μας έχει 4 cores χωρίς hyperthreading. Από την άλλη στα Pthreads είδαμε και κει μία αισθητή βελτίωση ίσως λόγο απόκρυψης χρόνου πρόσβασης στην μνήμη. Έπειτα από μία έρευνα βρήκαμε ότι η openMP πολλές φορές μπορεί αν ζητήσω παραπάνω threads από τα physical cores να δώσει όσα είναι τα Physical.
“Normally, the OpenMP runtime automatically chooses the thread count such that all cores on your system receive one thread and the system is not overloaded ... OpenMP does not differentiate physical cores and logical cores available with Hyper-Threading.”

https://www.openmp.org/spec-html/5.0/openmpse51.html

Έπειτα θέσαμε την μεταβλητή περιβάλλοντος OMP_NUM_THREADS σε μεγαλύτερο αριθμό όπως αναφέρεται στο λινκ αλλά δεν υπήρξε κάποια αλλαγή.

Άσκηση 2.2

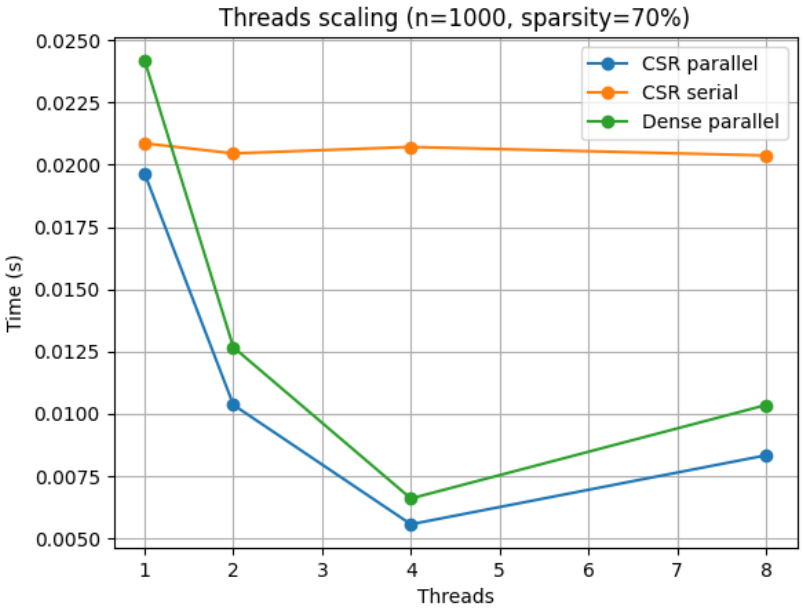
Το πρόβλημα του πολλαπλασιασμού πινάκων με διάνυσμα, επαναληπτικά (το vector αποτελέσματος γίνεται το vector εισόδου στην επόμενη επανάληψη) λύθηκε ως εξής:

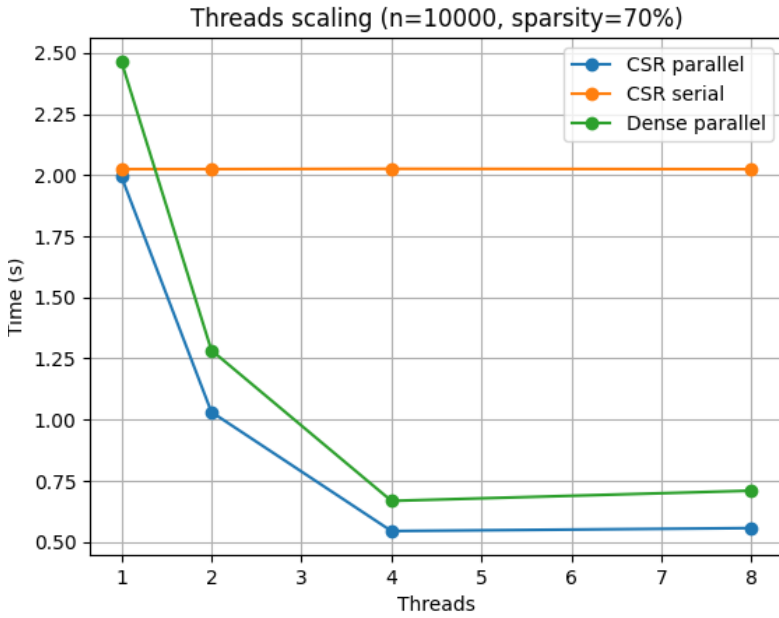
Αρχικά, αρχικοποιούνται ο πίνακας και το vector με τυχαίους αριθμούς. Έπειτα γίνεται η αρχικοποίηση των δομών CSR με παράλληλο τρόπο όμως υπάρχει ένα σημείο σειριοποίησης, όταν πρέπει να μετρήσουμε το πόσα μη μηδενικά στοιχεία υπάρχουν μέχρι αυτήν την γραμμή (εξάρτηση από την προηγούμενη γραμμή). Μετράμε παράλληλα το πόσα στοιχεία έχει κάθε γραμμή , αφού δεν υπάρχει εξάρτηση μεταξύ διαφορετικών γραμμών, και το μόνο που γίνεται σειριακά είναι ένα for όσο είναι το dimension.

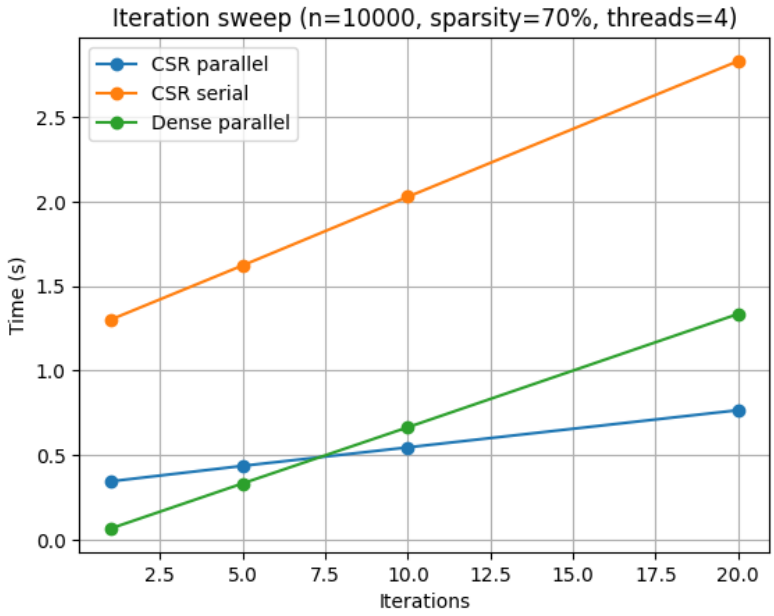
Μία αλλαγή που κάναμε ήταν ότι κρατάγαμε έναν counter για το πόσα non zero elements υπάρχουν αλλά υπολογίζονται στην τυχαία αρχικοποίηση του πίνακα στην αρχή οπότε ουσιαστικά δεν προσμετρόταν στο χρόνο αρχικοποίησης των δομών csr. Τελικά το βάλαμε πιο κάτω σε ένα παραλληλοποιημένο for.

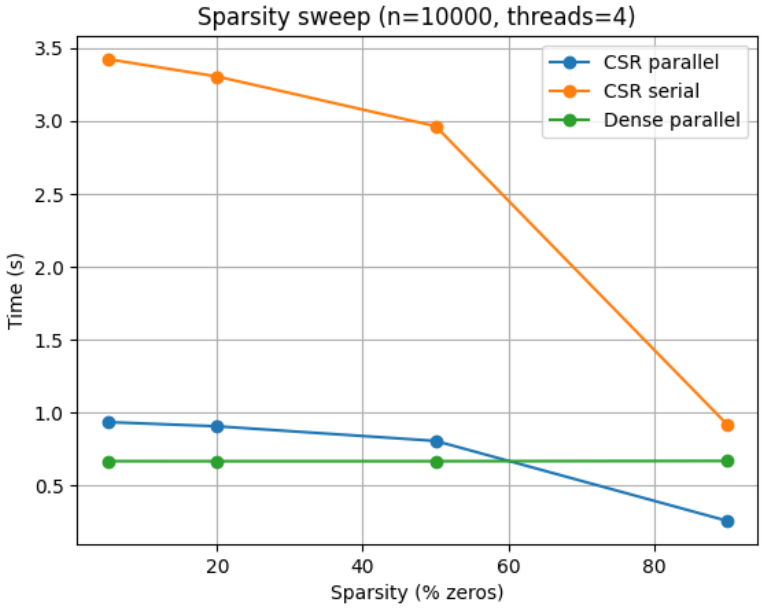
Μία ακόμη δοκιμή που κάναμε ήταν στην δημιουργία των πινάκων CSR (έχοντας υπολογίσει τον πίνακα row_indeces) να τσεκάρουμε αν έχουμε φτάσει τα ακριβή μηδενικά στοιχεία (row_indeces[i + 1] - row_indeces[i]) ώστε να γίνεται break μην συνεχίζεται το loop αφού δεν υπάρχουν άλλα μηδενικά στοιχεία σε αυτήν την σειρά. Παρόλα αυτά υπήρξε μία μικρή επιβάρυνση στον χρόνο και το αφαιρέσαμε (λογικά διότι ο έλεγχος σε σχέση με το πόσα loops γλιτώναμε δεν ήταν αποτελεσματικός σε χρόνο).

Παρακάτω παρατίθενται τα διαγράμματα, τα οποία τρέξαν για 4 φορές το κάθε πείραμα και με σταθερές 10000 γραμμές και στήλες κάθε πίνακας και 10 επαναλήψεις (για να είναι σχετικά βαρύ οπότε να είναι πιο εμφανείς οι χρόνοι) με 4 threads για να εκμεταλλευτούμε την παραλληλία και μηδενικά 70%.









<u>Σύγκριση παράλληλου CSR με Serial :</u>

Αρχικά, βλέπουμε ότι όσο αυξάνονται τα threads (εως 4 που είναι και τα physical cores) η παράλληλη υλοποίηση τα πάει εξαιρετικά αφου εκμεταλλεύται πλήρως την παραλληλία και στο initialization και στην εκτέλεση του πολλαπλασιασμού. Έπειτα βλέπουμε ότι για 8 threads υπάρχει μία μικρή επιβάρυνση η οποία όμως σε n=10000 αποκρύπτεται αφού ο φόρτος εργασίας είναι μεγαλύτερος και μέχρι και 2 threads ανα core μπορεί να βελτιώνουν λίγο την απόδοση λόγω απόκρυψης χρόνο στην ram η οποία εξισώνεται με τον χρόνο του context switch. Επίσης βλέπουμε οτί και για πολλά και για λίγα iteration η διαφορά χρόνου serial-parallel είναι σταθερή. Μία ενδιαφέρουσα παρατήρηση είναι ότι για πολύ μεγάλο αριθμό μηδενικών (90%) η διαφορά μεταξύ παράλληλου και σειριακού CSR μειώνεται

κατακόρυφα και αυτό εξηγείται αφού όσα περισσότερα μηδενικά τόσο λιγότερη η εργασία που γίνεται από τον CSR αλγόριθμο (δεν γίνεται πολλαπλασιασμός – προσπελάσεις στην μνήμη).

<u>Σύγκριση παράλληλου CSR με παράλληλο πυκνής αναπαράστασης :</u>

Βλέπουμε ότι ο παράλληλος αλγόριθμος πυκνής αναπαράστασης τα πάει εξίσου καλά σε κάποιες περιπτώσεις και μάλιστα για λιγότερο αραιούς πίνακες είναι καλύτερος πχ 40-50%, κάτι που είναι λογικό αφού η μέθοδος csr εκμεταλέυεται τα πολλά μηδενικά και μάλιστα βλέπουμε ότι για περισσότερα μηδενικά είναι καλύτερη. Για σταθερό αριθμό μηδενικών και επαναλήψεων η γραφική παράσταση της πυκνής αναπαράστασης είναι μετατοπισμένη κατακόρυφα – δηλαδή για κάθε αριθμό thread είναι ανάλογα χειρότερη. Παρομοίως για πολλές επαναλήψεις η μέθοδος CSR είναι καλύτερη αφού οι πράξεις που κάνει είναι λιγότερες και αυτό στις πολλές επαναλήψεις φαίνεται πιο έντονα (ότι δηλαδή αξίζει το overhead της δημιουργίας ως προς την δουλειά που εν τέλη μας γλιτώνει η μέθοδος)

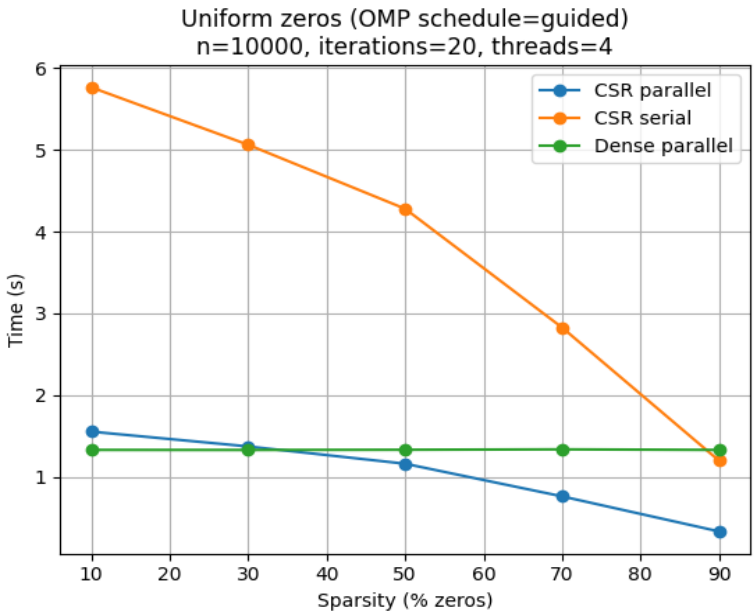
<u>Άνιση κατανομή 0:</u>

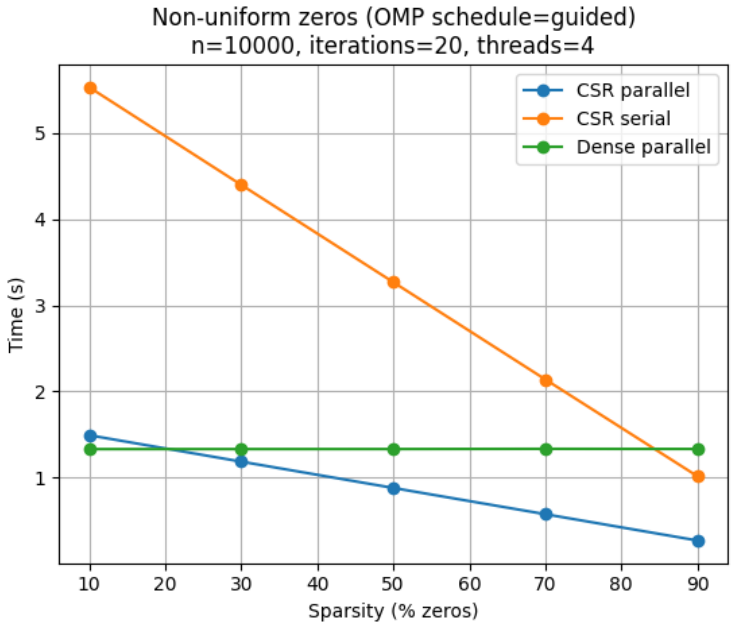
Με την παράμετρο Y(yes) στο πρόγραμμα, βάζει όλα τα μηδενικά του πίνακα (όσα είναι αυτά ανάλογα το ποσοστό) στην αρχή του πίνακα. Παρακάτω υπάρχουν τα διαγράμματα για άνιση και φυσιολογική κατανομή μηδενικών με static και dynamic scheduling αντίστοιχα.

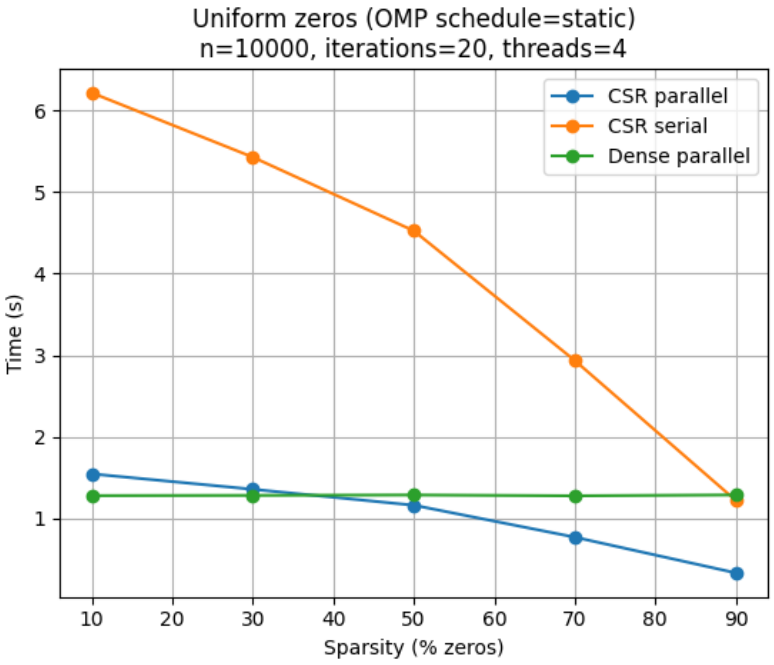
<u>Προσοχή :</u> Το scheduling στο πρόγραμμα έχει οριστεί ως runtime για να μπορεί να αλλάζει από την μεταβλητή περιβάλλοντος. Αν τρέξετε το πρόγραμμα μέσω του script τότε δεν υπάρχει πρόβλημα γιατί ορίζει την μεταβλητή κατάλληλα.

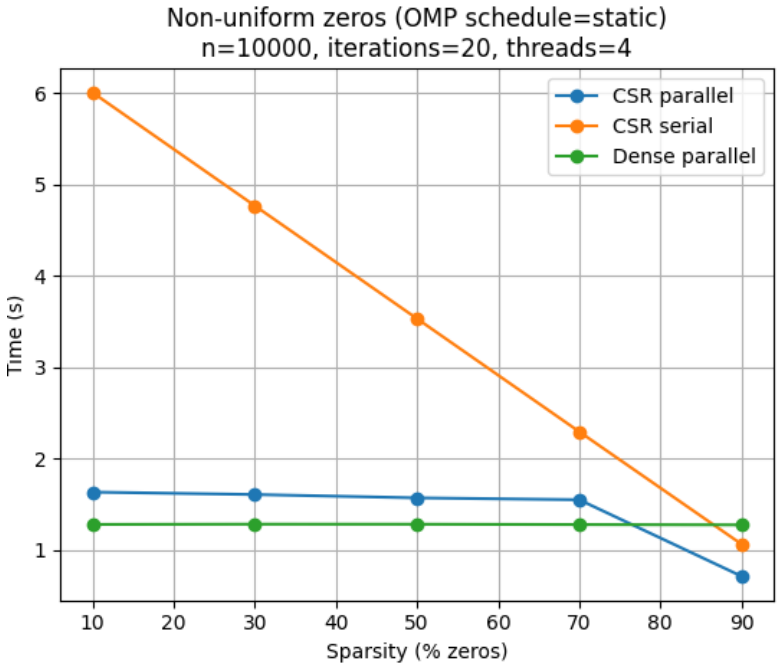
Αν όμως τρέξει χειροκίνητα ο χρήστης είναι υπεύθυνος να έχει ορίσει σωστά την μεταβλητή περιβάλλοντος (οι μετρήσεις έχουν γίνει με static).

Όπως βλέπουμε στα διαγράμματα για uniform κατανομή δεν υπάρχει διαφορά μεταξύ static και guided, αλλά όταν είναι non uniform η παραλληλία με guided λειτουργεί πολύ καλύτερα (ειδικά για 70% μηδενικά βλέπουμε 50% βελτίωση). Αυτό συμβαίνει γιατί όταν η κατανομή είναι άνιση κάποια threads που έχουν μόνο μηδενικά δεν έχουν ουσιαστικά δουλειά γιατί όλες οι γραμμές που έχουν πάρει είναι γεμάτες μηδενικά. Έτσι όταν έχουμε static και ο πίνακας απλά σπάει σε 4 κομμάτια δεν θα έχουν όλα τα threads την ίδια δουλειά, κάτι που διορθώνεται με το guided (ή και το static,2 πχ που οι χρόνοι είναι ίδιοι με το guided και δεν μπήκαν τα διαγράμματα για αυτόν τον λόγο).


Uniform zeros (OMP schedule=guided)
n=10000, iterations=20, threads=4
CSR parallel
CSR serial
Dense parallel
Time (s)
Sparsity (% zeros)


Non-uniform zeros (OMP schedule=guided)
n=10000, iterations=20, threads=4
CSR parallel
CSR serial
Dense parallel
Time (s)
Sparsity (% zeros)


Uniform zeros (OMP schedule=static)
n=10000, iterations=20, threads=4
CSR parallel
CSR serial
Dense parallel
Time (s)
Sparsity (% zeros)


Non-uniform zeros (OMP schedule=static)
n=10000, iterations=20, threads=4
CSR parallel
CSR serial
Dense parallel
Time (s)
Sparsity (% zeros)

Άσκηση 2.3

Το πρόβλημα αυτό αναδεικνύει την επιτάχυνση ενός αλγορίθμου ταξινόμησης με τη παραλληλοποίηση του προγράμματος με τη χρήση OpenMP και ειδικότερα με τη χρήση tasks. Ο αλγόριθμος merge-sort υλοποιείται και σειριακά και παράλληλα με τη διαφορά ελάχιστων μόνο εντολών. Το ζητούμενο είναι να αρχικοποιηθεί ένας πίνακας του επιθυμητού μεγέθους με ντετερμινιστικό τρόπο, ώστε για σταθερό μέγεθος οι τιμές του πίνακα να μην μεταβάλλονται. Στη συνέχεια, ο πίνακας αυτός ταξινομείται με την ταξινόμηση merge-sort με την οποία, ο πίνακας κάθε φορά υποδιαιρείται μέχρις ότου φτάσει να είναι ένα ατομικό στοιχείο και αναδρομικά κατά την συγχώνευση των πινάκων ταξινομούνται τα στοιχεία.

Όμοια με τον κώδικα των διαφανειών για το πρόβλημα Fibonacci, ένα μόνο thread πρέπει να ξεκινάει την αναδρομική κλήση του αλγορίθμου που αφορά το κομμάτι της υποδιαίρεσης του πίνακα και που δημιουργεί τα αρχικά tasks. Έπειτα της δημιουργίας των task αυτών πρέπει να γίνει ένα taskwait προκειμένου να γίνει η συγχώνευση μόνο αφού θα έχουν ετοιμαστεί οι δύο υποπίνακες.

Το κρίσιμο σημείο στο πρόβλημα αυτό είναι η επιλογή ενός ελάχιστου μεγέθους πινάκων για το οποίο ο παράλληλος αλγόριθμος θα δημιουργεί επιπλέον tasks. Με λίγα λόγια, χρειάζεται να ορίσουμε μία συνθήκη μέσω του if() clause, όπου για πάνω από ένα συγκεκριμένο μέγεθος πινάκων θα δημιουργούνται tasks που παρέχουν τη δυνατότητα εκτέλεσης παράλληλων εργασιών. Αντιθέτως εάν το μέγεθος του πίνακα δεν ξεπερνάει τον αριθμό αυτόν, το κομμάτι κώδικα του task εκτελείται σειριακά.

Το ελάχιστο μέγεθος πίνακα για το οποίο δημιουργούνται tasks, έχει οριστεί ως το 10000. Ο αριθμός αυτός επιλέχθηκε πειραματικά εφόσον για μικρότερα νούμερα, ο χρόνος εκτέλεσης του παράλληλου αλγορίθμου υπερβαίνει αυτόν του σειριακού. Αυτό οφείλεται στο ότι το overhead για την δημιουργία και υλοποίηση των task είναι πολύ μεγάλο και δεν αξίζει για μικρά μεγέθη πίνακα που μπορούν να ταξινομηθούν γρήγορα και σειριακά.

Παρακάτω παρουσιάζονται τα αποτελέσματα, ενώ οι χρόνοι που δίνονται, προκύπτουν από τον Μ.Ο 5 εκτελέσεων.

Μέγεθος πίνακα: 10^7

Αλγόριθμος	**Πλήθος νημάτων**			
	1	2	4	8
Παράλληλος	2.402481 seconds	1.298218 seconds	.752433 seconds	.764353 seconds
Σειριακός	2.074647 seconds			

Επιτάχυνση S = Tserial / Tparallel

	Πλήθος νημάτων			
	1	2	4	8
S	0.86	1.6	2.75	2.71

Μέγεθος πίνακα: 10^8

Αλγόριθμος	**Πλήθος νημάτων**			
	1	2	4	8
Παράλληλος	25.952939 seconds	13.927043 seconds	7.910858 seconds	8.074823 seconds
Σειριακός	22.743228 seconds			

Επιτάχυνση

	Πλήθος νημάτων			
	1	2	4	8
S	0.87	1.63	2.87	2.81

Όπως αναφέρθηκε και προηγουμένως, η επιτάχυνση του προγράμματος ήταν εφικτή μόνο στην περίπτωση που δεν δημιουργούνται απεριόριστα tasks για όλες τις αναδρομικές κλήσεις ανεξαρτήτως μεγέθους πινάκων. Αυτό επιταχύνει σημαντικά τον παράλληλο αλγόριθμο όπως αναδεικνύεται από τα αποτελέσματα, αφού για «μικρά» μεγέθη εκτελείται σειριακά και για μεγάλα παράλληλα. Για την εκτέλεση με ένα thread, ο χρόνος ξεπερνάει ελάχιστα αυτόν του σειριακού αφού στην πραγματικότητα δεν έχουμε παραλληλία παρά μόνο overhead για να δημιουργηθεί το νήμα αυτό. Επιπλέον, με την αύξηση των threads μειώνεται ο χρόνος εκτέλεσης όπως είναι αναμενόμενο αφού, περισσότερα νήματα μπορούν να εκτελέσουν παράλληλα διαφορετικά tasks. Προφανώς, από 4 σε 8 threads δεν παρουσιάζεται κάποια βελτίωση, αφού ο αριθμός των πυρήνων είναι ο μισός και τα threads στην πραγματικότητα δεν μπορούν να εκτελέσουν τη δουλεία τους εντελώς ταυτόχρονα λόγω των context switch. Το speedup παραμένει παρόμοιο και για τα δύο μεγέθη πινάκων, με το 10^8 να παρουσιάζει μία ελάχιστη βελτίωση.